



Explainable e-sports win prediction through Machine Learning classification in streaming[☆]

Silvia García-Méndez^{ID*}, Francisco de Arriba-Pérez

Information Technologies Group, atlantTic, University of Vigo, Vigo, Spain

ARTICLE INFO

Keywords:

Artificial Intelligence
e-sports
Explainability
Machine Learning
Real-time data analytics
Win prediction

ABSTRACT

The increasing number of spectators and players in e-sports, along with the development of optimized communication solutions and cloud computing technology, has motivated the constant growth of the online game industry. Even though Artificial Intelligence-based solutions for e-sports analytics are traditionally defined as extracting meaningful patterns from related data and visualizing them to enhance decision-making, most of the effort in professional winning prediction has been focused on the classification aspect from a batch perspective, also leaving aside the visualization techniques. Consequently, this work contributes to an explainable win prediction classification solution in streaming in which input data is controlled over several sliding windows to reflect relevant game changes. Experimental results attained an accuracy higher than 90%, surpassing the performance of competing solutions in the literature. Ultimately, our system can be leveraged by ranking and recommender systems for informed decision-making, thanks to the explainability module, which fosters trust in the outcome predictions.

1. Introduction

Electronic sports (*i.e.*, e-sports) are one of the world's most international and popular sportive events, involving millions of users [1]. The increasing number of spectators and players in e-sports, along with the development of optimized communication solutions and cloud computing technology, has motivated the constant growth of the online game industry [2] and the increasing popularity of Artificial Intelligence (AI) based solutions for e-sports analytics [3]. Notably, the commercial relevance of professional winning prediction is remarkable. In this line, note that this is the most watched part of the game commentator show [4].

Most e-sports, particularly regarding the multiplayer online battle arena (MOBA) e-sports (*e.g.*, Counter-Strike: Global Offensive - CS: GO, Defense of the Ancients 2 - DOTA 2, Heroes of Newerth - HON, Honor of Kings - HOK, League of Legends - LOL, Tactical Troops: Anthracite Shift - TT: AS) attract millions of players and thus, produce a publicly available stream of data regarding matches and competitions. Researchers can easily store and retrieve this vast amount of data to perform e-sports analytics, which can provide valuable tactical knowledge to players [2]. Regarding AI-based e-sports systems, there exist solutions to advise players on attack tactics and provide recommendations to

win the game [5], among other valuable applications like tournament classification exploiting Machine Learning (ML) models [6].

Explainable AI (XAI) was created with the ultimate objective of providing transparency to the above ML models that operate as black boxes (*i.e.*, those whose working is unknown by the end users) [7]. Regarding XAI, it can be performed following different approaches: (i) counterfactual explanations [8], (ii) feature importance [9,10], (iii) natural language descriptions [11], and (iv) visualization [12]. Compared to interpretability, which focuses on self-explanatory techniques (*e.g.*, intrinsic interpretable tree-based models), explainability refers to the use of post-hoc techniques, mainly model-agnostic [13]. Thus, explainability (*e.g.*, local linear and random models like LIME and SHAP [14]) is mostly directed to black-box classification algorithms. Ultimately, the ML models must provide good results and comprehensive interpretation [15]. However, even though e-sports analytics is traditionally defined as extracting meaningful patterns from related data and visualizing them to enhance decision-making, most of the effort has been focused on the classification part, overlooking the visualization techniques [4,16]. Moreover, the latter classification task is also unexplored on a stream basis.

[☆] This work was partially supported by (i) Xunta de Galicia, Spain grants ED481B-2022-093 and ED481D 2024/014, Spain; and (ii) University of Vigo/CISUG for open access charge.

* Corresponding author.

E-mail addresses: sgarcia@gti.uvigo.es (S. García-Méndez), farriba@gti.uvigo.es (F.d. Arriba-Pérez).

Consequently, the lack of interpretability and explainability is one of the most significant limitations of previous works focusing on pre-match prediction, which cannot utilize live data streams as input data for the models [17]. Ultimately, we apply explainability techniques to promote accountability, fairness, interoperability, reliability, and trustworthiness of our solution [18]. By these means, we also increase acceptance and adoption of Δ_1 by stakeholders, policymakers, online users, and society. Another relevant limitation is that these solutions utilize data from a vast temporal space, which contrasts with the dynamic nature of e-sports. Compared to traditional sports, the core mechanics of e-sports can be significantly altered over time due to continuous updates, making past data obsolete.

In light of the above, the ultimate objective of this work is to create an ML-based win prediction solution for e-sports. Given the characteristics of the use case, the system must operate in real time, and explanations of its operation must be provided to ensure its accountability and reliability and promote its acceptance among end-users. Accordingly, input data is controlled over several sliding windows to reflect relevant game changes. In fact, dynamic data management using sliding windows has been previously applied to different use cases in e-sports apart from the MOBA genre, which endorses its adaptability and appropriateness [19–21].

Notably, our work focuses on the tactical first-person shooter video game CS: GO, for which players must exhibit good coordination with their team members, low reaction times, and an effective economic strategy to win [22]. It represents an appropriate game for e-sports analytics due to its complex round-based mechanics (e.g., map biases, the existence of ties compared to binary-outcome e-sports) that allow studying the interplay of short- and long-term decisions during the game. More specifically, two teams play on either the *counter-terrorist* (CT) or *terrorist* (T) side. Each team comprises five players, and each event consists of 30 rounds with a time limit, starting with a predetermined amount of money. After round 15, the economies are reset since the teams switch sides to reduce the imbalance caused by the map choice. Note that in the start round (i.e., the *pistol round*), the money given to the players is limited, and the latter grows when killing enemies, defusing, or planting a bomb. The team that wins 16 rounds wins the game. In the event of a tie, and if the competition allows it, overtime exists, typically consisting of 6 additional rounds, with map switches occurring every 3 rounds. Overtime ends when a team wins 4 out of 6 rounds in the overtime stage. To increase the value of our proposal, the system was designed to minimize the training data required for win prediction (approximately half the game).

To summarize, the primary motivation of our work is its application to ranking and recommender systems, which necessitates the existence of an explainability module to promote trust in the outcome predictions. The main objectives of this study are:

- Application of streaming techniques for win prediction in e-sports, avoiding the popular, costly batch processing strategy.
- Generation of a wide range of features to model the evolving nature of e-sport users' behavior and patterns, including analyzing historical data.
- Creation of natural language descriptions about the predictive behavior of the models included in a graphical dashboard to assist players in real time.
- The possibility of an expert-in-the-loop to verify and assess the explainability outputs to promote the system's accountability.

The rest of this paper is organized as follows. Section 2 overviews the relevant competing win prediction systems in the literature regarding the application domain. Section 3 introduces the proposed solution, while Section 4 describes the experimental dataset, implementations, and set-up, along with the empirical evaluation results. Finally, Section 5 concludes and highlights the achievements and future work.

2. Related work

In electronic entertainment, e-sports have become viral, thanks to the broadcasting of professional competitions whose audience is constantly rising [23]. Accordingly, CS: GO has recently broken its record player count, reaching 1 802 853 concurrent players on May 2023.¹ Consequently, e-sports is relevant to the industry and the research community due to its significant commercial size and data value [17, 24].

Among the most popular applications of e-sports analytics, win prediction, ranking of players [25], and prediction of players' skills [26] deserve special mention. Win prediction in e-sports is a non-straightforward task that can be addressed from different perspectives [27]. Some authors address it as a binary classification problem (i.e., lose or win); others, like a regression problem in which a constant value is predicted based on the input features [28]. Regarding input data, existing solutions exploit a wide range of features (e.g., historical match data, player data, and team features) [29]. Primarily, it involves skill modeling, which is relevant to e-sport matchmaking and understanding player collaboration. In this line, note its direct application in ranking players and teams in professional competitions based on historical data for game commentator use cases [30].

In light of the above, ML is an appropriate approach for e-sports analytics due to the availability of high-dimensional (pre-game, in-game, and post-game features related to the match, players, and teams) and high-volume data from MOBA e-sports [31]. However, there is still a need to analyze the input features and their relevance towards enhanced win prediction [28,32]. Moreover, previous win prediction solutions do not investigate how to interpret the classification outcomes [16]. Consequently, the lack of interpretability dramatically limits the application of e-sports win prediction solutions in live streaming. Among the features used in e-sports research, i.e., pre-game, in-game, and post-game features, the in-game ones stand out [17].

Pre-game features are those extracted from data before the match starts.

In-game features reflect how the game evolves and are usually exploited for real-time prediction.

Post-game features summarize the match, often at the end of the game, indicating the damages, rewards, and match duration, among other relevant features.

Table 1 summarizes the ML-based MOBA win prediction solutions from the literature. Note that all works surveyed follow the supervised approach, except for the semi-supervised solution by Bisberg and Ferrara [29]. Only the system by Hodge et al. [17] applied real-time e-sports analytics. Furthermore, in-game features are broadly used to train the ML models. Finally, note that although few works provide explainability, the authors exclusively explore the feature importance approach [4,15,16,23,27,31,33].

As in our case, Makarov et al. [34] conducted experiments with a CS: GO dataset. Additional experiments were performed with DOTA 2 data. Moreover, the authors compared their decision tree-based win prediction model with TrueSkill, a Bayesian rating system.² Similar proposals for the DOTA 2 game are those by Stanly et al. [38], Uddin et al. [39]. Consideration should be given to the work by Kim et al. [35], who developed a novel confidence calibration method that takes into account data uncertainty. Conversely, Xiao et al. [36] designed WP-GBDT, which exploits an ensemble approach combining averaging and stacking for win prediction with gradient boosting.

¹ Available at <https://www.dexerto.com/csgo/how-many-people-play-csgo-player-count-record-2071859>, October 2025.

² Available at <https://trueskill.org>, October 2025.

Table 1

Comparison of ML-based MOBA win prediction solutions taking into account the e-sports domain, data processing (offline, online), features involved (pre-, in-, post-game), and explainability capability.

Authorship	Domain	Process	Features	Explainability
Makarov et al. [34]	DOTA 2	Offline	In-game	✗
Hodge et al. [17]	CS: GO	Online	In-game	✗
Kim et al. [35]	DOTA 2	Offline	In-game	✗
Xiao et al. [36]	LOL	Offline	In-game	✗
Bahrololloomi et al. [27]	TT: AS	Offline	In-game	✗
Birant and Birant [31]	LOL	Offline	In-game	Feature importance
			Post-game	Feature importance
Bisberg and Ferrara [29]	LOL	Offline	In-game	✗
Hitar-Garcia et al. [23]	LOL	Offline	Pre-game	Feature importance
Shen [37]	LOL	Offline	In-game	✗
Stanlly et al. [38]	DOTA 2	Offline	Pre-game	✗
Tian et al. [4]	HOK	Offline	In-game	Feature importance
Uddin et al. [39]	DOTA 2	Offline	In-game	✗
Yang et al. [16]	HOK	Offline	In-game	Feature importance
Bahrololloomi et al. [33]	LOL	Offline	In-game	Feature importance
Boidot et al. [15]	LOL	Offline	In-game	Feature importance
Proposed	CS: GO	Online	Pre-game	Feature importance and path
			In-game	Natural language description
			Post-game	Visual dashboard

Furthermore, Birant and Birant [31] proposed a Multi-Objective Multi-Instance Learning (MOMIL) for win prediction. The authors explored multi-instance learning (MIL) with five feature vectors for each of the five players in the team, and if they win the game, the sample is associated with the target value 1; otherwise, it is associated with the target value 0. The authors used entropy to measure the impurity of the season-based training data. In contrast, Bisberg and Ferrara [29] created GCN-WP, an e-sport semi-supervised win prediction solution based on neighborhood classification with convolutional neural networks. Similar proposals are those by Hitar-Garcia et al. [23], Shen [37].

In favor of interpretability, Birant and Birant [31] employed the OneR algorithm to assess features' importance according to the minimum error rate on the experimental data. Comparably, feature importance computation by Hitar-Garcia et al. [23] is based on a decision tree model. Furthermore, Tian et al. [4] proposed a Hero Featured Network (HFN) that exploits player information. Since the model used is opaque, the Long Short Term Memory network (LSTM), the authors decouple the contributions of the input features. In this line, Yang et al. [16] proposed an interpretable win prediction system exploiting a two-stage spatial-temporal network (TSSTN) to extract the contribution of the input features to the final prediction. More recently, Boidot et al. [15] evaluated explanations generated with SHAP (a post hoc feature importance explanation method derived from game theory [40]) on an LOL win prediction use case, similar to the earlier proposals by Bahrololloomi et al. [27,33].

No related work has applied data management with sliding windows. Outside the MOBA genre, representative examples are the solutions by Spijkerman and van der Haar [41] and Hojaji et al. [42] for the specific case of Formula One and simulated racing, respectively. Like our work, Scotti et al. [43] utilized data management with sliding windows, in addition to feature engineering, for stream decoding in football matches.

In light of the above, our work addresses the research gap of jointly considering online analytics and providing explainability in win prediction.

3. System architecture

Fig. 1 shows the scheme of the solution. It comprises in-game data fusion (Section 3.1), stream-based data processing (Section 3.2), stream-based classification (Section 3.3), and post-game stream-based explainability (Section 3.4).

3.1. In-game data fusion

For in-game data fusion, the event identifier, map played, and timestamp will be used to identify a game uniquely and the 5 players per team that participated. The data from each of the 5 players in each team will be aggregated. The latter is especially relevant in e-sports like CS: GO, where each player depending on their spam (*i.e.*, where the player appears at the start of the round), weapons, cumulative economy, and abilities, must assume a different role (*entry frager, support, lurker*, etc.) and therefore their skills will be affected. A snapshot of the player's skills at the team level is computed thanks to the fusion process.

3.2. Stream-based data processing

Data processing is essential to ensure the effective and efficient performance of the ML models in classification problems, even more so in the case of streaming operation. Thus, the system integrates (i) pre-game feature engineering to enhance performance and (ii) feature analysis and selection to promote efficiency in the proposed solution.

3.2.1. Pre-game feature engineering

Players have an individual evolutionary profile that depends on their seniority level (*i.e.*, the number of games played). For this reason, the system builds upon accumulated knowledge over past samples. These samples will be added and removed dynamically, employing sliding windows to compute the distribution function of the number of games per player. The latter computation requires: (i) an initial chunk of the streaming data to avoid cold start, (ii) the calculation of the number of games per player for that subset, (iii) the values of average, 25th, 50th and 75th percentiles (equivalent to the first quartile or q1, q2, and q3) of the game distribution function.

More in detail, four sliding windows will be applied, each with the samples equivalent to the values mentioned above (*i.e.*, average, 25th, 50th, and 75th percentiles). For each of the four sliding windows and each feature in the experimental dataset, some values will be calculated: average, 25th, 50th, and 75th percentiles, the standard deviation (STD), and the maximum modulus of the Fast Fourier Transform (FFT). Resulting in 6 values per feature and window. It is important to note that not all players will have as much past information as the number of inputs needed for each window. In this case, the window size will be calculated as the minimum between the window size itself and the number of features. Fig. 2, Algorithm 1, and Algorithm 2 illustrate the feature engineering process using sliding windows.

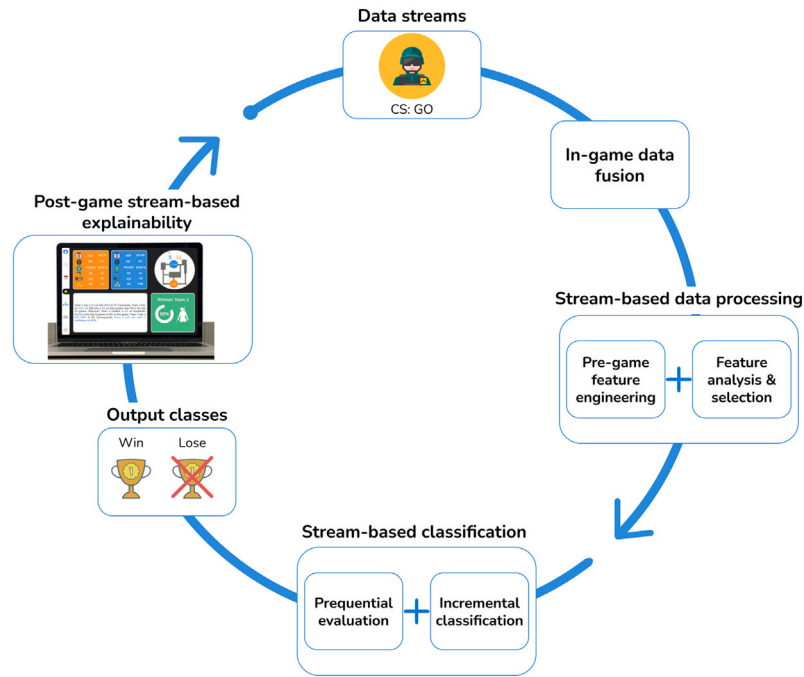


Fig. 1. System diagram composed of: (i) in-game data fusion, (ii) stream-based data processing, (iii) stream-based classification, and (iv) post-game stream-based explainability.

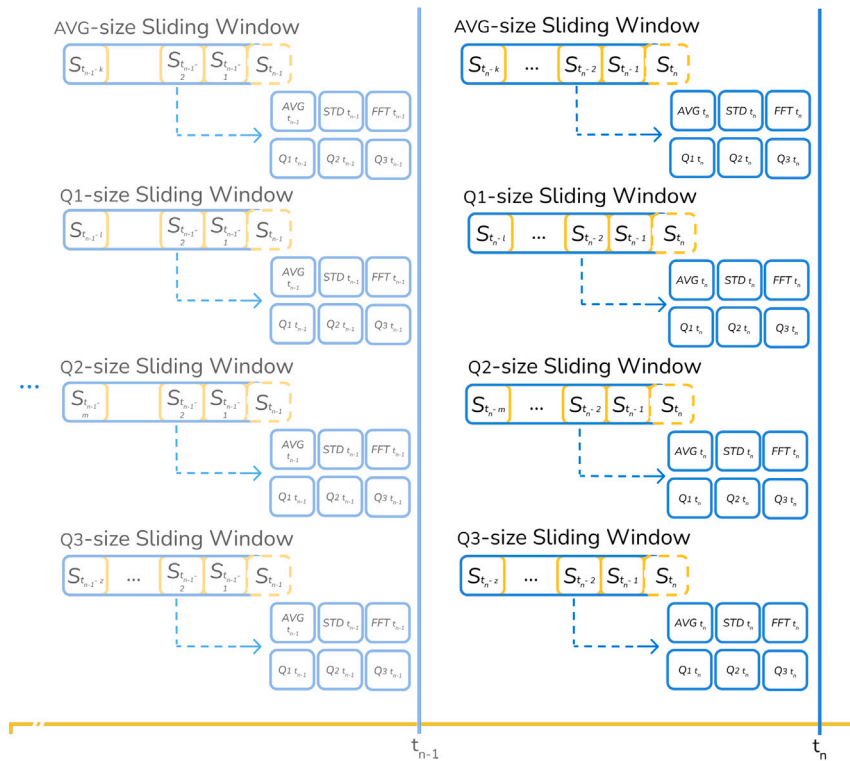


Fig. 2. Features engineered using the sliding windows.

Algorithm 1: Windows size adjustment.

Input: Players history.
Data: *player_game_history_matrix*
number_games_vector=[]
for *player_id* **in** *set(player_game_history_matrix["Player ID"])* **do**
 player_data = *player_game_history_matrix["Player ID"].filter(player_id)*
 number_games_vector.expand(len(player_data)) // This calculates the number of times a player has played a game and adds it to the vector.
average_window = *number_games_vector.mean()*
q1_window = *number_games_vector.percentil(25)*
q2_window = *number_games_vector.percentil(50)*
q3_window = *number_games_vector.percentil(75)*
return [*average_window, q1_window, q2_window, q3_window*] // Returns four windows sizes for feature engineering.

Algorithm 2: Feature engineering.

Input: Player ID, player history, and windows sizes.
// This method is executed for each player involved in a game.
Data: *player_id, player_game_history_matrix, average_window, q1_window, q2_window, q3_window*
player_data = *player_game_history_matrix["Player ID"].filter(player_id)*
statistic_features_vector=[]
for *window* **in** [*average_window, q1_window, q2_window, q3_window*] **do**
 if *window* > *len(player_data)* **then**
 window = *len(player_data)*
 for *column* **in** *player_data.columns* **do**
 column_values_vector = *player_data[column][0:window]*
 [*average, maximum, minimum, q1, q2, q3, std, f ft*] = *feature_statistics_eq1(column_values_vector)* // The method of calculation is described in Equation 1.
 statistic_features_vector.expand(mean, maximum, minimum, q1, q2, q3, std, f ft)
return *statistic_features_vector* // Returns all the features generated for each statistic and sliding window.

Eq. (1) shows how the statistic values for the features are computed, where n is the number of games of a player and $X[n]$ is the historical feature in the last n player games. Consequently, $Y[n]$ is the ordered version of $X[n]$.

$$\begin{aligned}
 &\forall n \in \{1 \dots \infty\} \\
 &X[n] = \{x[0], \dots, x[n]\}. \\
 &Y[n] = \{y_0[n], y_1[n], \dots, y_{n-1}[n]\} \mid y_0[n] \leq y_1[n] \leq \dots \leq y_{n-1}[n], \\
 &\text{where; } \forall x \in X[n], x \in Y[n]. \\
 &\text{avg}[n] = \frac{1}{n} \sum_{i=0}^n y_i[n] \\
 &Q_1[n] = y_{\lfloor \frac{1}{4}n \rfloor}[n] \\
 &Q_2[n] = y_{\lfloor \frac{2}{4}n \rfloor}[n] \\
 &Q_3[n] = y_{\lfloor \frac{3}{4}n \rfloor}[n] \\
 &\text{std}[n] = \sigma(X[n]) \\
 &F[n] = |FFT(X[n])|_{\infty}
 \end{aligned} \tag{1}$$

3.2.2. Feature analysis & selection

Reducing the number of features used for training the ML models is vital in every classification problem [44]. The latter becomes essential in the stream-based scenario to ensure the effective and efficient performance of the proposed solution [45]. Moreover, the selected features may vary since the system operates in a streaming environment.

Low-relevant features will be removed, exploiting the variance thresholding approach for feature selection based on variance analysis. Thus, features with low variance are discarded, inspired by the literature, which considers these features to be lowly relevant for classification purposes [46]. Firstly, the variance of the features in each incoming sample is computed. Only the features that meet a configurable threshold are considered in the following stages. More in detail, the variance threshold is obtained using the cold start subset mentioned in Section 3.2.1 as required in the streaming scenario. Then, the variance of the features in that subset is calculated. Finally, the value of the 10th percentile is selected as the threshold, indicating the value from which the feature should be considered an outlier.

3.3. Stream-based classification

Fig. 3 shows the supervised classification pipeline in streaming and batch. In contrast to batch learning, the first step involves feature selection, and the selector update occurs with every incoming sample, ordered by their timestamp. Ultimately, the sample is classified, and the model is updated according to this current sample. The main difference between the two approaches is that the prediction in batch occurs after selector update and feature extraction. Conversely, the prediction comes first in stream-based classification, and then the model is re-trained. Additionally, in the batch scheme, the training process involves the entire dataset, which prevents the model from undergoing a scalable, continuous update.

Consequently, in our study, the ML models are first tested using prequential evaluation and then trained on the incoming samples. Note that data decimation is applied in training and testing; one in ten samples is selected, and the rest are discarded.

Below, the ML models used for classification are listed. Note that these models were selected inspired by the revision of the competing works in the literature [15,17,33,34]. Moreover, deep learning models were excluded due to the limitations related to fixed network weights after training, which prevents their application in a streaming scenario [47] and its black-box nature since they are not intrinsically interpretable like tree-based models [48].

- **Gaussian Naive Bayes (GNB)** [49]. It is based on the traditional Gaussian distribution from the Naive Bayes (NB) model, which is appropriate for stream-based classification. It is used as a baseline.
- **Hoeffding Adaptive Tree Classifier (HATC)** [50]. This model is based on an online single tree with a mechanism to evaluate branch performance.
- **Adaptive Random Forest Classifier (ARFC)** [51]. It is similar to the latter model but employs an ensemble strategy that comprises resampling and random feature selection. Prediction is obtained by computing majority voting.

3.4. Post-game stream-based explainability

For end-users in a real-time gaming context, explainability can help them be accountable for their performance, anticipate potential

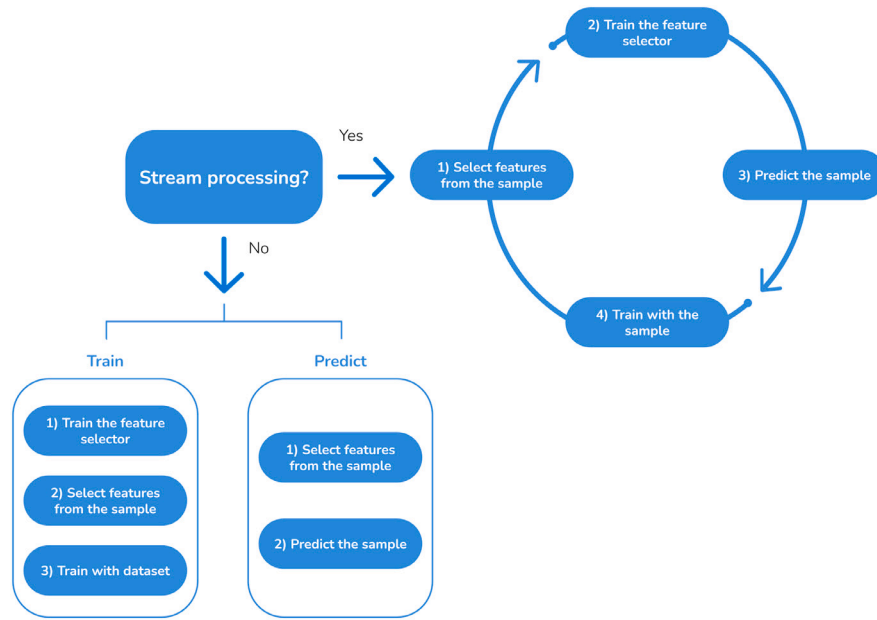


Fig. 3. Streaming and batch supervised classification.

Algorithm 3: Explainability process.

Input: Estimator and sample.
 // This process is repeated for each estimator in the tree classifier.
Data: estimator, sample
begin
 feature_frequency_vector = []
 tree_node = estimator[0]; // Root node.
 while tree_node ≠ nil **do**
 feature = tree_node[feature]
 threshold = tree_node[threshold]
 feature_frequency_vector.append(feature)
 if sample[feature] ≤ threshold **then**
 | tree_node = tree_node[left_branch]
 else
 | tree_node = tree_node[right_branch]
 feature_frequency_vector = feature_frequency_vector.counter().values()
return feature_frequency_vector[0 : 5] // Returns first 5 features sorted by frequency of occurrence.

rewards, or be prepared for penalties in sporting events. The same applies to identifying areas for improvement in the game for future events and detecting risky patterns and behaviors that may lead to losing the match or tournament. At the same time, the explanations provided may help e-sport users optimize the resources available (e.g., team selection and economic strategy). In short, χ_{AI} can promote player reflection, learning, and immersion in the game.

Accordingly, post-game features are exploited as part of the information provided to the end users in the explainability dashboard. To gather the relevant data to be included in the natural language descriptions, the decision path (*greater than* and *lower than* branches) is traversed to extract the track and the feature occurrence number (i.e., frequency of the features). A single or configurable number of estimators is considered for the HATC and the ARFC models, respectively. The latter is used to order the features to select the most important for classification purposes. Moreover, the performance of the ML model is also included as part of the natural language description and explainability dashboard using the statistical parameters regarding the sliding windows (average, Q_1 , Q_2 , Q_3 , STD and FFT signal). Algorithm 3 summarizes the process.

4. Evaluation and discussion

All the experiments were executed on a computer with the following specifications:

- **Operating System:** Ubuntu 18.04.2 LTS 64 bits
- **Processor:** Intel@Core i9-10900K 2.80 GHz
- **RAM:** 96 GB DDR4
- **Disk:** 480 GB NVME + 500 GB SSD

4.1. Experimental dataset

The publicly available dataset³ comprises information about the players' skills (subset P, 383 317 samples) and the results of the cs: go (subset R, 45 773 samples), in separate csv files. For clarity, note that an event comprises up to 3 games (in different maps) lasting between 16 and 30 rounds.

Each sample in the subset P constrains players' metrics regarding each event (see Table 2, raw features). Conversely, each sample in the subset R represents a game between two teams (see Table 3, raw features). Summarizing, the experimental data are organized at the team (team 1 and team 2) and side (CT and T) levels. Using the feature #7 in Table 3, the data in both subsets were assigned to a specific team and part of the game (first or second half). If team 1 or 2 starts in the CT side in the first half of the game, it will play in the T side in the second half. Features #11 and #12 in Table 2 and feature #11 in Table 3 are engineered with the latter transformation.

³ Available at <https://www.kaggle.com/datasets/mateusdmachado/csgo-professional-matches>, October 2025.

Table 2Features in subset P by type (raw from the original dataset or engineered — Engi.).

Num.	Type	Name	Description
1	Raw	Date	Date of the event.
2		Event ID	ID of the event.
3		Game ID	ID of the game.
4		Team ID	ID of the team.
5		Player ID	ID of the player.
6		Map{1,2,3}	ID of the map.
7		Skills	Basic & advanced skills for all maps.
8		Skills by side{CT,T}	Basic skills for all maps by side.
9		Skills by map{1,2,3}	Basic & advanced skills in the map.
10		Skills by map{1,2,3} and side{CT,T}	Basic skills in the map by side.
11	Engi.	Skills by half{1,2}	Basic skills for all maps by game half.
12		Skills by map{1,2,3} and half{1,2}	Basic skills in the map and game half.

Table 3Features in subset R by type (raw from the original dataset or engineered — Engi.).

Num.	Type	Name	Description
1	Raw	Date	Date of the event.
2		Event ID	ID of the event.
3		Game ID	ID of the game.
4		Team ID	ID of the team.
5		Map{1,2,3}	ID of the map.
6		H1TV rank by team{1,2}	Team Half-Life Television ranking ^a .
7		Starting CT side	Team ID in the CT side in the first half.
8		Score by team{1,2}	Number of rounds won by team.
9		Score by team{1,2} and side{CT,T}	Number of rounds won by team and side.
10		Target	The ID of the team that won the game.
11	Engi.	Score by team{1,2} and half{1,2}	Number of rounds won by team and half.

^a Available at <https://www.hltv.org>, October 2025.**Table 4**

Distribution of classes in the experimental dataset.

Winner	Number of samples
Team 1	24 236
Team 2	20 843
Total	45 079

Moreover, the players' skills comprise:

Basic skills: (i) average damage per round (ADR), (ii) the number of deaths (ND), (iii) percent of rounds the player assisted, killed, survived or was traded (PRAKST), (iv) the number of kills (NK), (v) difference between deaths and kills (K/D DIFF), (vi) rating in the game (PR).

Advanced skills: (i) the number of assists (NASS), (ii) the number of flash assists (NFA), (iii) the number of headshots (NHS), (iv) difference of the cumulative first to kill and first to die per round (FK/FD DIFF).

Samples in both subsets were selected based on their date, resulting in 376 469 and 45 079 samples for the subset P and R , respectively. The number of players involved is 12 153, and the average of games per player is 3.71. Table 4 summarizes the distribution of the samples per target class in our classification problem.

4.2. In-game data fusion

To fusion the in-game features in the P and R subsets, the features #1, #2, #3, and #6 / #5 (map ID) in Tables 2 and 3 are used as the

key. The latter selection results in 10 (5 players in each team) and 1 sample of the P and R subsets, respectively.

To extract the skills related to each team, we used the feature #4 in both Tables 2 and 3 as key, and then applied Eq. (2) using feature #12 in Table 2 and feature #11 in Table 3 (h1 and h2 stand for the first and second half of the game).

$$skills_{team} = feature\#12(h1) + \frac{feature\#12(h2)}{feature\#11(h2)} \quad (2)$$

4.3. Stream-based data processing

In this section, the implementations used and the results obtained for (i) pre-game feature engineering and (ii) feature analysis and selection stages are presented and discussed.

4.3.1. Pre-game feature engineering

The average, 25th, 50th, and 75th percentiles of the cold start data (4507 samples) correspond to 12, 2, 3, and 9. The latter values enable the system to engineer cumulative features for both basic and advanced players' skills (see feature #7 in Table 2). Fig. 4 shows the distribution function of the number of games per player, which follows an exponentially decreasing tendency.

Table 5
Features used for training and testing after data fusion and processing.

Num.	Type	Name	Description
1	In-game	Date	Date of the event.
2		Map{1,2,3}	ID of the map.
3		Score by team and side in 1st half	Number of rounds won by team and side in the first half of the game.
4		HLTV rank by team	Team Half-Life Television ranking.
5		Skills by team	Aggregation of basic skills in the map by side in the first 16 rounds.
6	Pre-game	Sliding windows skills	Sliding windows features of basic and advanced skills for all maps (average, 25th, 50th, 75th, STD and FFT).

The in-game variables obtained in the data fusion stage (see Section 3.1) are then merged with the pre-game features engineered (see Section 3.2.1), resulting in 279 features (39 in-game and 240 pre-game features). Table 5 summarizes the in-game and pre-game features used. These will be the input features used to train the stream-based classification models.

4.3.2. Feature analysis & selection

To evaluate the cumulative variance of the features engineered for each incoming sample, the solution employs `VarianceThreshold`⁴ from `River`.⁵ The selection threshold for scenario A and B, explained below, are 0.004 and 0.076, respectively.

4.4. Stream-based classification

Two experimental scenarios were designed.

- **Scenario A.** It comprises the features #1–#4 from Table 5 to evaluate the effect of in-game features regarding team data.
- **Scenario B.** It comprises all the features from Table 5 to evaluate the combined effect of pre-game and in-game data, including players' skills.

ML models were incrementally trained and evaluated using an *ad hoc* implementation of `EvaluatePrequential`.⁶

The online supervised classification models used are `GNB`,⁷ `HATC`⁸ and `ARFC`⁹ from `River`. Listings show the hyper-parameter optimization ranges used for the `HATC` and `ARFC` models, respectively. Note that the `GNB` model has no hyper-parameters to tune. The selected hyper-parameters are:

HATC: [50, 0.5, 50], [50, 0.5, 50] for scenario A and scenario B, respectively.

ARFC: [100, 50, 10], [50, 50, 10] for scenario A and scenario B, respectively.

Listing 1: HATC hyperparameter configuration.

```
1 depth = [50, 100, 200]
2 tiethreshold = 0.5, 0.05, 0.005]
3 maxsize = [50, 100, 200]
```

⁴ Available at <https://riverml.xyz/0.11.1/api/feature-selection/VarianceThreshold>, October 2025.

⁵ Available at <https://riverml.xyz/0.11.1>, October 2025.

⁶ Available at <https://scikit-multiflow.readthedocs.io/en/stable/api/generatetd/skmultiflow.evaluation.EvaluatePrequential.html>, October 2025.

⁷ Available at <https://riverml.xyz/dev/api/naive-bayes/GaussianNB>, October 2025.

⁸ Available at <https://riverml.xyz/0.11.1/api/tree/HoeffdingAdaptiveTreeClassifier>, October 2025.

⁹ Available at <https://riverml.xyz/0.11.1/api/ensemble/AdaptiveRandomForestClassifier>, October 2025.

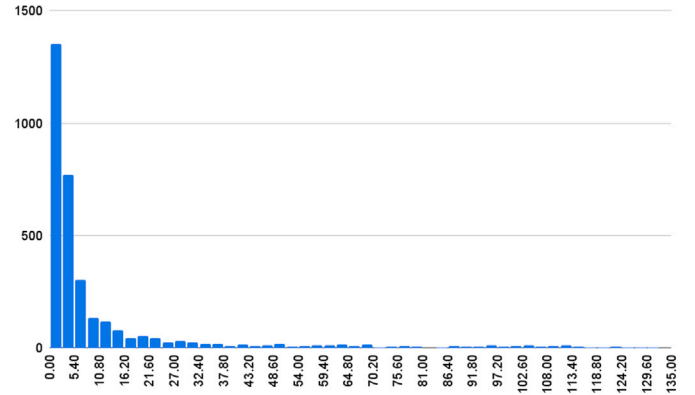


Fig. 4. Distribution function of the number of games per player.

Table 6 shows the results obtained with the ML models selected in the experimental scenarios mentioned above. For the first scenario, A, the performance difference between models is not statistically significant (*i.e.*, less than 5%). The evaluation metrics are promising between 68% and 79%. Moreover, in scenario B, the effect of combining pre-game and in-game features, including data related to players' skills, improves the results for `HATC` and `ARFC`. Unfortunately, the simplicity of the baseline `GNB` model prevents it from taking advantage of the latter features. In this regard, the biggest difference are between the `GNB` baseline and `ARFC` (*i.e.*, 18.34%, 16.98%, 17.37%, 18.33% in accuracy, precision, recall and *F*-measure, respectively). Since it is a baseline model, `GNB` is 40 times faster than `ARFC`. In the end, the `ARFC` classifier exhibits the best performance (*i.e.*, all metrics are above 90%). Even though it has a high computational cost, this does not affect the stream operation of the win prediction solution (*i.e.*, real-time processing is achieved since the model can process 22.5 samples per second). Note that the latter model is interpretable by traversing the branches of its decision trees.

Listing 2: ARFC hyperparameter configuration.

```
1 models = [25, 50, 100]
2 features = [50, 150, 300]
3 lambda = [10, 50, 100]
```

To further evaluate the robustness of the proposed streaming solution, we apply a segment-wise validation strategy following the nested cross-validation method used in continuous learning.¹⁰ For this purpose, the data stream is divided into sequential, non-overlapping blocks

¹⁰ Available at https://scikit-learn.org/stable/auto_examples/model_selection/plot_nested_cross_validation_iris.html, October 2025.

Table 6

Win prediction results in streaming.

Sec.	Model	Acc.	Precision			Recall			F-measure			Time (s)
			Macro	#1	#2	Macro	#1	#2	Macro	#1	#2	
A	GNB	74.97	74.84	75.85	73.82	74.59	79.08	70.10	74.67	77.43	71.91	1.15
	HATC	74.02	73.89	74.82	72.95	73.59	78.59	68.59	73.68	76.66	70.70	2.21
	ARFC	74.99	74.86	75.86	73.86	74.61	79.12	70.10	74.69	77.46	71.93	144.72
B	GNB	74.17	75.60	83.88	67.33	74.98	64.38	85.58	74.11	72.85	75.36	5.33
	HATC	84.61	84.51	85.91	83.11	84.54	85.41	83.67	84.53	85.66	83.39	13.00
	ARFC	92.51	92.58	91.87	93.30	92.35	94.44	90.25	92.44	93.14	91.75	200.58

Table 7

Win prediction results in streaming using nested cross-validation (scenario B).

Model	Acc.	Precision			Recall			F-measure			Time (s)	
		Macro	#1	#2	Macro	#1	#2	Macro	#1	#2		
GNB	74.12	74.43	79.69	69.18	74.49	69.65	79.33	74.12	74.34	73.90	4.13	
HATC		75.88	75.79	76.57	75.00	75.58	79.49	71.67	75.65	78.00	73.30	9.88
ARFC		88.87	88.93	88.44	89.42	88.68	91.25	86.10	88.78	89.82	87.73	170.33

Table 8

Performance comparison with the literature.

Authorship	Domain	Processing	Accuracy	Explainability
Makarov et al. [34]	CS: GO	Offline	62.00%	✗
Hodge et al. [17]	DOTA2	Online	77.51%	✗
Proposal	CS: GO	Online	92.51%	Feature importance and path Natural language description Visual dashboard

of 10000 instances. In each block, the classifier and feature selector are restarted, and their performance is evaluated following the prequential scheme (prediction before training). This procedure allows us to analyze the model's performance across the stream. As it can be observed in Table 7, ARFC is the best-performing classifier with values around 90 in all evaluation metrics, showing a performance reduction of less than 5% compared to traditional streaming evaluation in Table 6. This additional validation contributes to a more realistic assessment of system behavior when data is not available simultaneously but instead arrives incrementally and when models can only access partial historical data.

Finally, Table 8 summarizes the performance comparison between our solution and the literature. Only Makarov et al. [34] applied win prediction in CS: GO. The authors did not publicly provide their experimental dataset, which prevented evaluation of our solution, and Hodge et al. [17] operated on a stream-based approach. As a result of the theoretical comparison, our solution's accuracy exceeds 30.51 and 15 percent points for the above solutions, respectively.

Regarding the scalability of the proposed solution, it simulates streaming operations by following a single-thread scheme. Parallel computation, where several models simultaneously serve the requests, will considerably reduce its response time. Elastic hardware solutions, like distributed infrastructure, would be appropriate. Moreover, technologies such as Apache Kafka and Spark can also adequately manage the data stream. However, the replica models would not have a complete picture of the incoming data in this context. Alternatively, allocating threads could be based on specific competitions and platforms. More specifically, possible challenges lie in the heterogeneity of e-sports genres regarding performance indicators and game mechanics, which necessitate that the solution be able to integrate multimodal data in real time. About the APIs employed for data gathering, consideration should be given to their latency. Ultimately, scaling the solution across multiple games or platforms would benefit from its modular design.

4.5. Post-game stream-based explainability

Fig. 5 illustrates the explainability dashboard that summarizes the most relevant post-game features from the pre-game and in-game ones.

16-games 5-players accumulated feature values regarding the basic skills of the players are displayed at the top. PRAKST is calculated considering the 5 players. On the right, the scoreboard and map are displayed, showing the players' positions in real time. At the bottom, the most relevant features involved in the decision path of the ARFC model are included in a natural language description of the prediction, with its confidence. The latter is visually summarized in the bottom right part.

Moreover, the dashboard allows the display of the decision paths by the tree model (see Fig. 6). To ensure a user-friendly interface, the unexplored tree nodes are not shown, and those in the natural language description are in bold. Similarly, the decision tree paths that do not align with the majority-voted classification output are also removed and, thus, cannot be displayed by navigating with the left and right blue arrows.

To evaluate the quality and usefulness of the explanations provided, the end user can assess them (see Fig. 7). By clicking on the red cross, the human-in-the-loop indicates that the explanation is inadequate, while when the green tick is clicked, a 5-level Likert scale is shown for a more detailed evaluation.

5. Conclusions

Due to the massive and rapid expansion of the gaming industry, e-sports analytics plays a vital role in academia and industry. Accordingly, most e-sports, particularly regarding the MOBA, attract millions of players and, thus, produce a publicly available stream of data regarding matches and competitions. The vast amount of data produced can be easily stored and retrieved by researchers to perform e-sports analytics, providing valuable tactical knowledge to players. In this sense, ML is an appropriate approach for e-sports analytics due to the availability of high-dimensional and high-volume data from MOBA e-sports. However, the lack of interpretability and explainability is one of the most significant limitations of previous works focusing on pre-match prediction, which cannot utilize live data streams as input data for the models.

In light of the above, this work contributes an explainable win prediction classification solution for streaming, where input data is controlled over several sliding windows to reflect relevant game changes.



Fig. 5. Explainability dashboard exploiting post-game features and combining visualization and natural language.

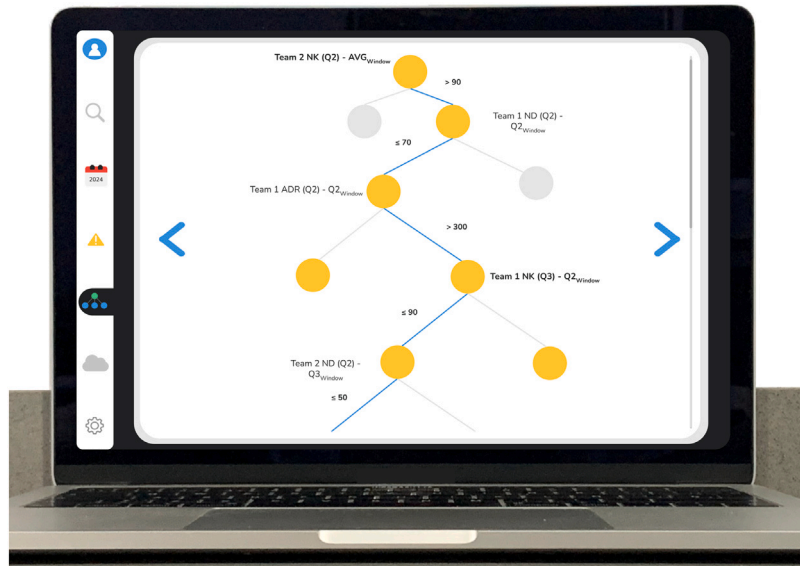


Fig. 6. Decision path displayed in the explainability dashboard.

Our solution addresses the research gap of jointly considering online analytics and providing explainability in win prediction. Experimental results attained an accuracy higher than 90% for win prediction in streaming.

Thanks to the explainability module, the system can be exploited by ranking and recommender systems for decision-making. This module promotes trust in the outcome predictions. Moreover, to increase the value of our proposal, the system was designed to minimize the training data needed (half of the game).

In future work, we plan to exploit live API information as input data to train our model and analyze how new features influence winning prediction (e.g., physiological features of players). Specifically, APIs like

Faceit,¹¹ Leetify,¹² Scope.gg¹³ and Steam¹⁴ can be used. Moreover, the system will be made available to the research community via API to analyze user feedback, notably regarding the explainability dashboard. Furthermore, new classification tasks (e.g., death prediction) and perspectives (e.g., unsupervised classification, reinforcement learning) will be taken into account. Finally, we will examine specific aspects of the solution, such as computing and storage load, to deploy it into production.

¹¹ Available at <https://docs.faceit.com>, October 2025.

¹² Available at <https://leetify.com>, October 2025.

¹³ Available at <https://scope.gg>, October 2025.

¹⁴ Available at <https://steamcommunity.com/dev>, October 2025.



Fig. 7. Explainability assessment.

CRedit authorship contribution statement

Silvia García-Méndez: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Francisco de Arriba-Pérez:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially supported by (i) Xunta de Galicia, Spain grants ED481B-2022-093 and ED481D2024/014, Spain; and (ii) University of Vigo/CISUG for open access charge.

Data availability

The used data is openly available. Refer footnote 3.

References

- [1] G. Pozzan, T. Vardanega, Rafting multiplayer video games, *Softw. Pr. Exp.* 52 (2022) 1065–1091, <http://dx.doi.org/10.1002/spe.3048>.
- [2] S.-S. Shin, Y.-H. Kim, Winner prediction for real-time strategy games through feature selection based on a genetic wrapper, in: *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, Association for Computing Machinery, 2021, pp. 65–66, <http://dx.doi.org/10.1145/3449726.3462735>.
- [3] M. Mora-Cantallos, M.-Á. Sicilia, MOBA games: A literature review, *Entertain. Comput.* 26 (2018) 128–138, <http://dx.doi.org/10.1016/j.entcom.2018.02.005>.
- [4] P. Tian, W. Lan, X. Zhang, Hero featured learning algorithm for winning rate prediction of honor of kings, in: *Proceedings of the IEEE Conference on Games*, 2022-August, IEEE, 2022, pp. 322–329, <http://dx.doi.org/10.1109/CoG51982.2022.9893634>.
- [5] F.-E. Ait-Bennacer, A. Aaroud, K. Akodadi, B. Cherradi, Applying deep learning and computer vision techniques for an e-sport and smart coaching system using a multiview dataset: Case of shotokan karate, *Int. J. Online Biomed. Eng. (Ijoeb)* 18 (2022) 35–53, <http://dx.doi.org/10.3991/ijoe.v18i12.30893>.
- [6] A.R. Dikananda, I. Ali, Fathurrohman, R.A. Rinaldi, Iin, Genre e-sport gaming tournament classification using machine learning technique based on decision tree, naïve Bayes, and random forest algorithm, *IOP Conf. Ser. Mater. Sci. Eng.* 1088 (2021) 012037–012043, <http://dx.doi.org/10.1088/1757-899X/1088/1/012037>.
- [7] M.A. Akbar, A.A. Khan, S. Mahmood, S. Rafi, S. Demi, Trustworthy artificial intelligence: A decision-making taxonomy of potential challenges, *Softw. Pr. Exp.* (2023) 1–30, <http://dx.doi.org/10.1002/spe.3216>.
- [8] S. Wachter, B. Mittelstadt, C. Russell, Counterfactual explanations without opening the black box: Automated decisions and the GDPR, *SSRN Electron. J.* (2017) 842–861, <http://dx.doi.org/10.2139/ssrn.3063289>.
- [9] U. Bhatt, A. Xiang, S. Sharma, A. Weller, A. Taly, Y. Jia, J. Ghosh, R. Puri, J.M.F. Moura, P. Eckersley, Explainable machine learning in deployment, in: *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, Association for Computing Machinery, 2020, pp. 648–657, <http://dx.doi.org/10.1145/3351095.3375624>.
- [10] Q.H. Vu, D. Ruta, L. Cen, M. Liu, A combination of general and specific models to predict victories in video games, in: *Proceedings of the IEEE International Conference on Big Data*, IEEE, 2021, pp. 5683–5690, <http://dx.doi.org/10.1109/BigData52589.2021.9671285>.
- [11] U. Ehsan, P. Tambwekar, L. Chan, B. Harrison, M.O. Riedl, Automated rationale generation: a technique for explainable AI and its effects on human perceptions, in: *Proceedings of the International Conference on Intelligent User Interfaces*, vol. Part F147615, Association for Computing Machinery, 2019, pp. 263–274, <http://dx.doi.org/10.1145/3301275.3302316>.
- [12] T. Spinner, U. Schlegel, H. Schafer, M. El-Assady, explAIner: A visual analytics framework for interactive and explainable machine learning, *IEEE Trans. Vis. Comput. Graphics* 26 (2019) 1064–1074, <http://dx.doi.org/10.1109/TVCG.2019.2934629>.
- [13] Y. Liu, X. Ding, S. Peng, C. Zhang, Leveraging ChatGPT to optimize depression intervention through explainable deep learning, *Front. Psychiatry* 15 (2024) 1383648, <http://dx.doi.org/10.3389/fpsy.2024.1383648>.
- [14] A.M. Salih, Z. Raisi-Estabragh, I.B. Galazzo, P. Radeva, S.E. Petersen, K. Lekadir, G. Menegaz, A perspective on explainable artificial intelligence methods: SHAP and LIME, *Adv. Intell. Syst.* (2024) 2400304, <http://dx.doi.org/10.1002/aisy.202400304>.
- [15] C. Boidot, O. Augereau, P.D. Loor, R. Lefort, Limits of XAI Application-Grounded Evaluation: An E-Sport Prediction Example, vol. 1752 CCIS, Springer, 2023, pp. 452–466, http://dx.doi.org/10.1007/978-3-031-23618-1_30.
- [16] Z. Yang, Z. Pan, Y. Wang, D. Cai, S. Shi, S.-L. Huang, W. Bi, X. Liu, Interpretable real-time win prediction for honor of kings—A popular mobile MOBA esport, *IEEE Trans. Game* 14 (2022) 589–597, <http://dx.doi.org/10.1109/TG.2022.3149044>.
- [17] V.J. Hodge, S. Devlin, N. Sephton, F. Block, P.I. Cowling, A. Drachen, Win prediction in multiplayer esports: Live professional match prediction, *IEEE Trans. Game* 13 (4) (2019) 368–379, <http://dx.doi.org/10.1109/TG.2019.2948469>.
- [18] S.G. Tesfagergish, R. Damaševičius, Explainable artificial intelligence for combating cyberbullying, in: *Communications in Computer and Information Science*, vol. 2030, Springer, 2024, pp. 54–67, http://dx.doi.org/10.1007/978-3-031-53731-8_5.

- [19] C.-M. Liaw, B.-R. Dai, Live stream highlight detection using chat messages, in: Proceedings of the IEEE International Conference on Mobile Data Management, IEEE, 2020, pp. 328–332, <http://dx.doi.org/10.1109/MDM48529.2020.00072>.
- [20] Y. Lu, H. Chen, H. Yan, E-sports competition analysis based on intelligent analysis system, Comput. Intell. Neurosci. 2022 (2022) 1–10, <http://dx.doi.org/10.1155/2022/4855550>.
- [21] M. Nicholson, D. Poulus, D. Johnson, R. Robergs, V. Kelly, C. McNulty, Role of a 10-week exercise intervention on cerebral hemoglobin saturation, cognitive function, and heart rate variability within elite esports players: A pilot study, J. Electron. Gaming Esports 2 (1) (2024) 1–14, <http://dx.doi.org/10.1123/jege.2024-0007>.
- [22] G. Brewer, S. Demediuk, A. Drachen, F. Block, T. Jackson, Creating well calibrated and refined win prediction models, SSRN Electron. J. (2022) <http://dx.doi.org/10.2139/ssrn.4054211>.
- [23] J.-A. Hitar-Garcia, L. Moran-Fernandez, V. Bolon-Canedo, Machine learning methods for predicting league of legends game outcome, IEEE Trans. Game (2022) 1–11, <http://dx.doi.org/10.1109/TG.2022.3153086>.
- [24] J.G. Reitman, M.J. Anderson-Coto, M. Wu, J.S. Lee, C. Steinkuehler, Esports research: A literature review, Game Cult. 15 (2020) 32–50, <http://dx.doi.org/10.1177/1555412019840892>.
- [25] J. Liu, J. Huang, R. Chen, T. Liu, L. Zhou, A two-stage real-time prediction method for multiplayer shooting E-sports, in: Proceedings of the International Conference on Electronic Business, 2020, pp. 9–18.
- [26] N. Khromov, A. Korotin, A. Lange, A. Stepanov, E. Burnaev, A. Somov, Esports athletes and players: A comparative study, IEEE Pervasive Comput. 18 (2019) 31–39, <http://dx.doi.org/10.1109/MPRV.2019.2926247>.
- [27] F. Bahrololloomi, S. Sauer, F. Klonowski, R. Horst, R. Dörner, A machine learning based analysis of e-sports player performances in league of legends for winning prediction based on player roles and performances, in: International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications, 2022, pp. 68–76, <http://dx.doi.org/10.5220/0010895900003124>.
- [28] L.M. Costa, R.G. Mantovani, F.C.M. Souza, G. Xexeo, Feature analysis to league of legends victory prediction on the picks and bans phase, in: Proceedings of the IEEE Conference on Games, vol. 2021-August, IEEE, 2021, pp. 1–5, <http://dx.doi.org/10.1109/CoG52621.2021.9619019>.
- [29] A.J. Bisberg, E. Ferrara, GCN-WP – semi-supervised graph convolutional networks for win prediction in esports, in: Proceedings of the IEEE Conference on Games, vol. 2022-August, IEEE, 2022, pp. 449–456, <http://dx.doi.org/10.1109/CoG51982.2022.9893671>.
- [30] A.J. Bisberg, R.E. Cardona-Rivera, Scope: Selective cross-validation over parameters for elo, in: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, vol. 15, 2019, pp. 116–122.
- [31] K.U. Birant, D. Birant, Multi-objective multi-instance learning: A new approach to machine learning for esports, Entropy 25 (2022) 1–18, <http://dx.doi.org/10.3390/e25010028>.
- [32] R. Ani, V. Harikumar, A.K. Devan, O. Deepa, Victory prediction in league of legends using feature selection and ensemble methods, in: Proceedings of the International Conference on Intelligent Computing and Control Systems, IEEE, 2019, pp. 74–77, <http://dx.doi.org/10.1109/ICCS45141.2019.9065758>.
- [33] F. Bahrololloomi, F. Klonowski, S. Sauer, R. Horst, R. Dörner, E-sports player performance metrics for predicting the outcome of league of legends matches considering player roles, SN Comput. Sci. 4 (2023) 1–12, <http://dx.doi.org/10.1007/s42979-022-01660-6>.
- [34] I. Makarov, D. Savostyanov, B. Litvyakov, D.I. Ignatov, Predicting Winning Team and Probabilistic Ratings in “Dota 2” and “Counter-Strike: Global Offensive” Video Games, vol. 10716 LNCS, Springer, 2018, pp. 183–196, http://dx.doi.org/10.1007/978-3-319-73013-4_17.
- [35] D.-H. Kim, C. Lee, K.-S. Chung, A confidence-calibrated MOBA game winner predictor, in: Proceedings of the IEEE Conference on Games, IEEE, 2020, pp. 622–625, <http://dx.doi.org/10.1109/CoG47356.2020.9231878>.
- [36] H. Xiao, Y. Liu, D. Du, Z. Lu, WP-GBDT: An approach for winner prediction using gradient boosting decision tree, in: Proceedings of the IEEE International Conference on Big Data, IEEE, 2021, pp. 5691–5698, <http://dx.doi.org/10.1109/BigData52589.2021.9671688>.
- [37] Q. Shen, A machine learning approach to predict the result of league of legends, in: Proceedings of the International Conference on Machine Learning and Knowledge Engineering, IEEE, 2022, pp. 38–45, <http://dx.doi.org/10.1109/MLKE55170.2022.00013>.
- [38] Stanly, F.A. Putra, N.N. Qomariyah, DOTA 2 win loss prediction from item and hero data with machine learning, in: Proceedings of the IEEE International Conference on Industry 4.0, Artificial Intelligence, and Communications Technology, IEEE, 2022, pp. 204–209, <http://dx.doi.org/10.1109/IAICT55358.2022.9887525>.
- [39] J. Uddin, I. Fahmida, S.I. Moyeen, M.M. Hasan, DOTA2 winner team prediction based on stacked bidirectional lstm network, in: Proceedings of the International Conference on Electrical, Computer & Telecommunication Engineering, IEEE, 2022, pp. 1–5.
- [40] S. Lundberg, S. Lee, A unified approach to interpreting model predictions. In: Advances in neural information processing systems, Adv. Neural Inf. Process. Syst. 32 (2017) 4765–4774.
- [41] R. Spijkerman, D. van der Haar, Video footage highlight detection in formula 1 through vehicle recognition with faster R-CNN trained on game footage, in: Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 12334 LNCS, Springer, 2020, pp. 176–187, http://dx.doi.org/10.1007/978-3-030-59006-2_16.
- [42] F. Hojaji, A.J. Toth, J.M. Joyce, M.J. Campbell, An AI approach for analyzing driving behaviour in simulated racing using telemetry data, in: Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 14475 LNCS, Springer, 2024, pp. 194–203, http://dx.doi.org/10.1007/978-3-031-49065-1_19.
- [43] V. Scotti, L. Sbattella, R. Tedesco, SPERANET: Automatic generation of football highlights, in: Proceedings of the International Conference on Computer Science, Engineering and Information Technology, Aircr Publishing Corporation, 2019, pp. 101–116, <http://dx.doi.org/10.5121/csit.2019.91309>.
- [44] A. Shaddeli, F.S. Gharehchopogh, M. Masdari, V. Solouk, BFRA: A new binary hyper-heuristics feature ranks algorithm for feature selection in high-dimensional classification data, Int. J. Inf. Technol. Decis. Mak. 22 (2023) 471–536, <http://dx.doi.org/10.1142/S0219622022500432>.
- [45] H.M. Gomes, J. Read, A. Bifet, J.P. Barddal, J. Gama, Machine learning for streaming data: state of the art, challenges, and opportunities, ACM SIGKDD Explor. Newsl. 21 (2019) 6–22.
- [46] A. Treisman, D. Mughaz, A. Stulman, A. Dvir, Word embedding dimensionality reduction using dynamic variance thresholding (DyVaT), Expert Syst. Appl. 208 (2022) 118157–118171, <http://dx.doi.org/10.1016/j.eswa.2022.118157>.
- [47] N.E. Protonotarios, I. Katsamenis, S. Sykiotis, N. Dikaos, G.A. Kastis, S.N. Chatziioannou, M. Metaxas, N. Doulamis, A. Doulamis, A few-shot U-net deep learning model for lung cancer lesion segmentation via PET/CT imaging, Biomed. Phys. Eng. Express 8 (2022) 025019, <http://dx.doi.org/10.1088/2057-1976/ac53bd>.
- [48] T. Qamar, N.Z. Bawany, Understanding the black-box: towards interpretable and reliable deep learning models, PeerJ Comput. Sci. 9 (2023) e1629, <http://dx.doi.org/10.7717/peerj-cs.1629>.
- [49] E. Tieppo, J.P. Barddal, J.C. Nievola, Classifying Potentially Unbounded Hierarchical Data Streams with Incremental Gaussian Naive Bayes, vol. 13073 LNAI, Springer, 2021, pp. 421–436, http://dx.doi.org/10.1007/978-3-030-91702-9_28.
- [50] Z.E. Mrabet, D.F. Selvaraj, P. Ranganathan, Adaptive hoeffding tree with transfer learning for streaming synchrophasor data sets, in: Proceedings of the IEEE International Conference on Big Data, IEEE, 2019, pp. 5697–5704, <http://dx.doi.org/10.1109/BigData47090.2019.9005720>.
- [51] H.K. Fatlawi, A. Kiss, On Robustness of Adaptive Random Forest Classifier on Biomedical Data Stream, vol. 12033 LNAI, Springer, 2020, pp. 332–344, http://dx.doi.org/10.1007/978-3-030-41964-6_29.